CC BY NC SA



UNIVERSIDAD DEL QUINDÍO ®
Res.MEN 014915 - 02 AGO 2022
RENOVACIÓN ACREDITACIÓN

Observabilidad y Monitoreo

Arquitectura Basada en Microservicios

Área de infraestructura
Programa de Ingeniería de Sistemas y Computación
Facultad de Ingeniería

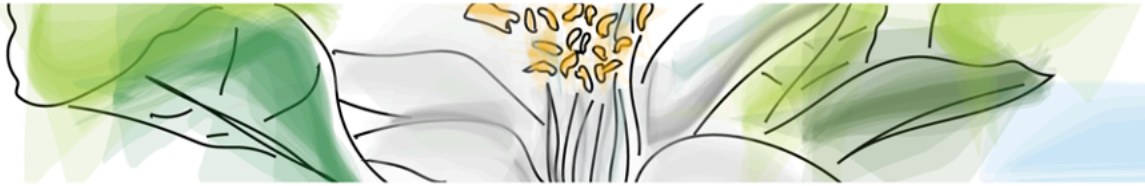
UNIQUINDÍO
en conexión territorial
www.uniquindio.edu.co





Contenido

- ¿Qué es la Observabilidad?
- SLI, SLO y SLA: el marco de los objetivos
- Los tres pilares: Métricas, Logs y Trazas
- Patrón: Health Check API
- Patrón: Health Check por Terceros
- Patrón: Distributed Tracing
- Patrón: Application Metrics
- Patrón: Audit Logging
- Patrón: Exception Tracking
- Patrón: Log Aggregation
- OpenTelemetry: el estándar
- Herramientas del ecosistema
- Alertas proactivas
- Mejores prácticas y antipatrones



¿Por qué necesitamos Observabilidad?

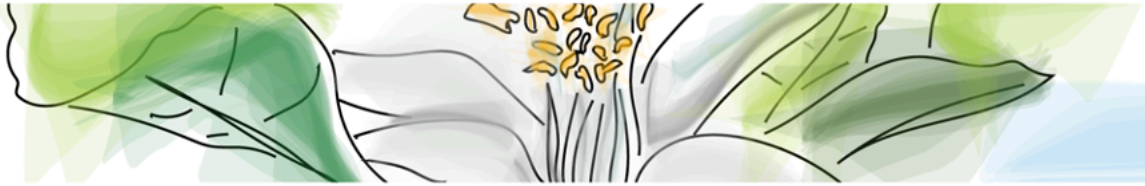
El problema: La caja negra

En un monolito cuando algo falla:

- El error está en **un solo proceso**
- Los logs están en **un solo lugar**
- El stack trace apunta al **archivo exacto**

En microservicios:

```
Cliente → Gateway → Empleados → Departamentos → Auth → Broker → ...
                            ↑
                ¿Dónde falló exactamente?
                ¿Cuánto tardó cada servicio?
                ¿Qué petición lo causó?
```



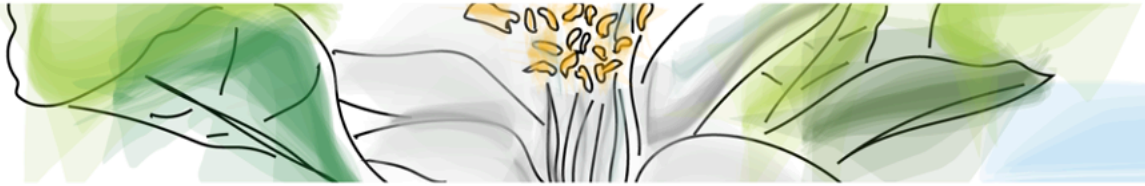
Observabilidad vs. Monitoreo

Aspecto	Monitoreo	Observabilidad
Pregunta	¿Está funcionando?	¿Por qué no funciona?
Enfoque	Métricas predefinidas	Estado interno inferido
Alcance	Comportamientos conocidos	Comportamientos inesperados
Acción	Reactiva	Proactiva y diagnóstica

> **Definición**: Un sistema es *observable* si su estado interno puede inferirse a partir de sus salidas externas: métricas, logs y trazas.



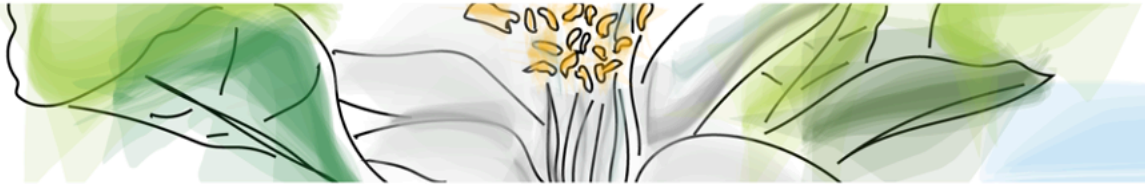
SLI, SLO y SLA: El marco de los objetivos



SLI, SLO y SLA

Concepto	Significado	Ejemplo concreto
SLI *(Indicator)*	Qué se mide	Latencia p99 del endpoint `/empleados`
SLO *(Objective)*	El objetivo interno del equipo	Latencia p99 < 300ms el **99.9%** del tiempo
SLA *(Agreement)*	El contrato externo con el cliente	Disponibilidad ≥ **99.5%** mensual

> Las **reglas de alerta** en Grafana deben derivarse de los SLOs, no ser arbitrarias.
> Si el SLO es "latencia < 300ms", la alerta se configura a 280ms para preservar el **error budget**.



SLI, SLO y SLA - Error Budget

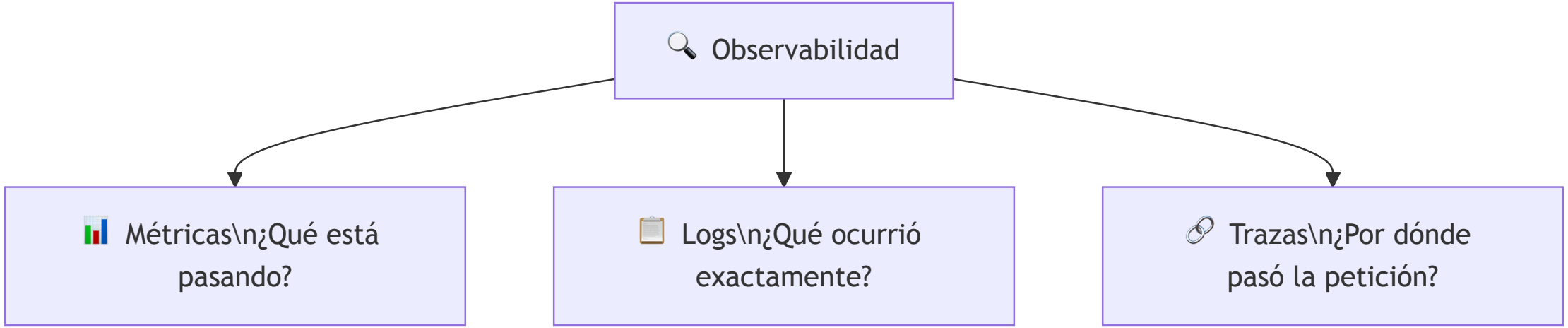
```
SLO: 99.9% de disponibilidad mensual
  → Se permiten 43 minutos de caída al mes (error budget)

Cuando el error budget se consume al 50%:
  → El equipo recibe alerta "Warning" → revisa el servicio

Cuando el error budget se consume al 100%:
  → Alerta "Critical" → congelar despliegues, priorizar estabilidad
```

> **Beneficio clave**: Conecta las métricas técnicas con decisiones de negocio. El equipo sabe cuánto margen tiene antes de impactar al cliente.

Los tres pilares de la Observabilidad



UNIQUINDÍO
en conexión territorial

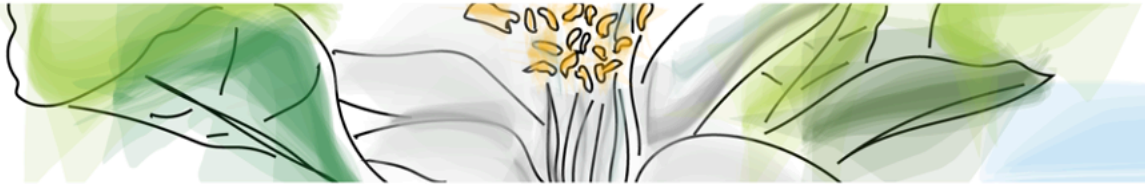


Pilar 1: Métricas

- **Definición**: Valores numéricos que representan el estado del sistema a lo largo del tiempo.

Tipos fundamentales:

Tipo	Descripción	Ejemplo
Counter	Solo incrementa	Total de peticiones
Gauge	Sube y baja	Conexiones activas
Histogram	Distribución de valores	Tiempo de respuesta
Summary	Percentiles calculados	p95 de latencia



Pilar 2: Logs

- **Definición**: Registros textuales de eventos discretos en el sistema.
- **Problema en microservicios**: Dispersos en múltiples servicios y contenedores.

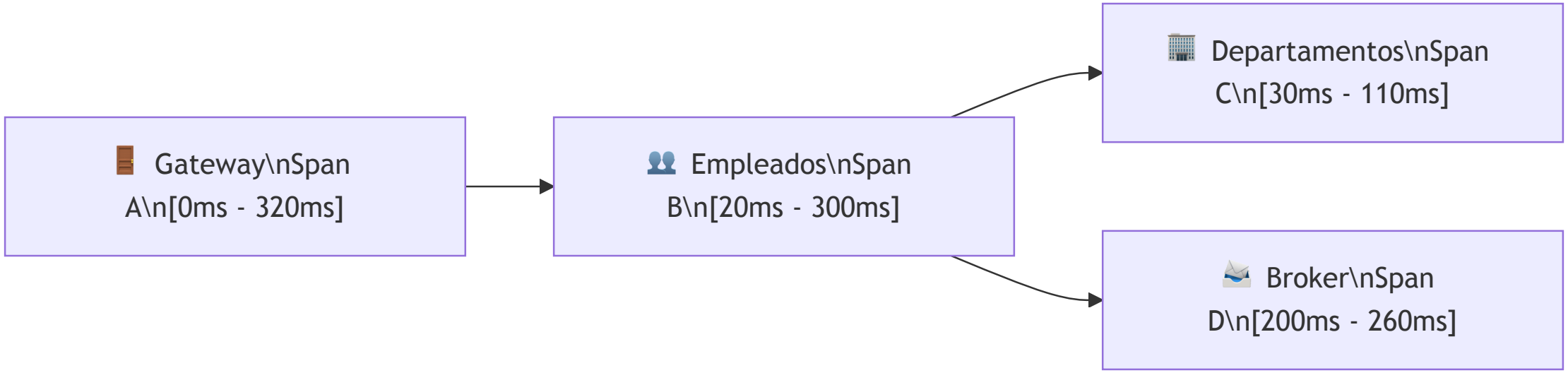
Log estructurado (recomendado):

```
{
  "timestamp": "2026-04-21T20:00:00Z",
  "level": "ERROR",
  "service": "empleados-service",
  "traceId": "abc123def456",
  "spanId": "789ghi",
  "message": "Departamento IT no encontrado"
}
```

Clave: El `traceId` conecta el log con la traza distribuida.

Pilar 3: Trazas Distribuidas

- **Definición**: Registro del recorrido completo de una petición a través de múltiples servicios.
- **Trace**: Toda la transacción de extremo a extremo (`traceId`)
- **Span**: Unidad de trabajo dentro del trace (cada servicio o llamada)



UNIQUINDÍO
en conexión territorial



Patrón: Health Check API

Health Check API

- **Problema**: ¿Cómo saber si un microservicio está funcionando correctamente y listo para recibir peticiones?
- **Posible solución**: Implementar un endpoint dedicado (`/health`) que exponga el estado operativo del servicio y de sus dependencias.

Health Check API - Tipos de probes

Tipo	Endpoint	Propósito
Liveness	`/health/live`	¿Está el proceso vivo? Si falla → reiniciar
Readiness	`/health/ready`	¿Listo para recibir tráfico? Si falla → sacar del balanceo
Startup	`/health/startup`	¿Terminó de inicializar?

Health Check API - Respuesta estándar

```
{
  "status": "UP",
  "service": "empleados-service",
  "version": "1.2.0",
  "checks": {
    "database": "UP",
    "messageBroker": "UP",
    "diskSpace": "UP"
  }
}
```

Código HTTP:

- `200 OK` → Estado `UP`
- `503 Service Unavailable` → Estado `DOWN`

Health Check API - Implementación por lenguaje

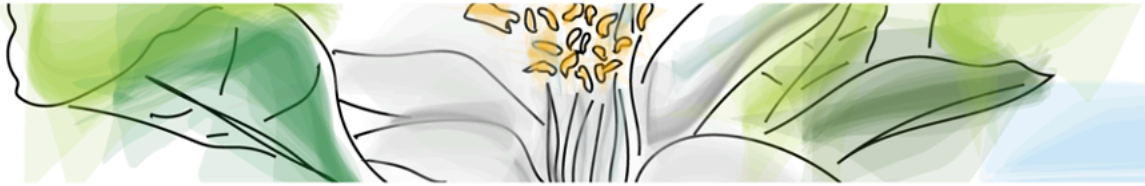
Lenguaje	Mecanismo
Java / Spring Boot	`/actuator/health` vía Spring Boot Actuator
Node.js / Express	Endpoint manual o librería `@godaddy/terminus`
Python / FastAPI	Endpoint manual verificando BD y broker
Python / Flask	Endpoint manual
Go	Endpoint manual o `alexliesenfeld/health`
.NET	`AddHealthChecks()` en ASP.NET Core

Health Check API - Beneficios y Dificultades

- **Beneficios**
  - Permite a orquestadores tomar decisiones automáticas (reinicio, balanceo)
  - Detecta degradaciones parciales (BD caída, broker no disponible)
  - Base para dashboards de estado en tiempo real
- **Dificultades**
  - Implementar checks de dependencias puede ser costoso
  - Un health check pesado puede degradar el propio servicio
  - Riesgo de falsos positivos si el check no es representativo



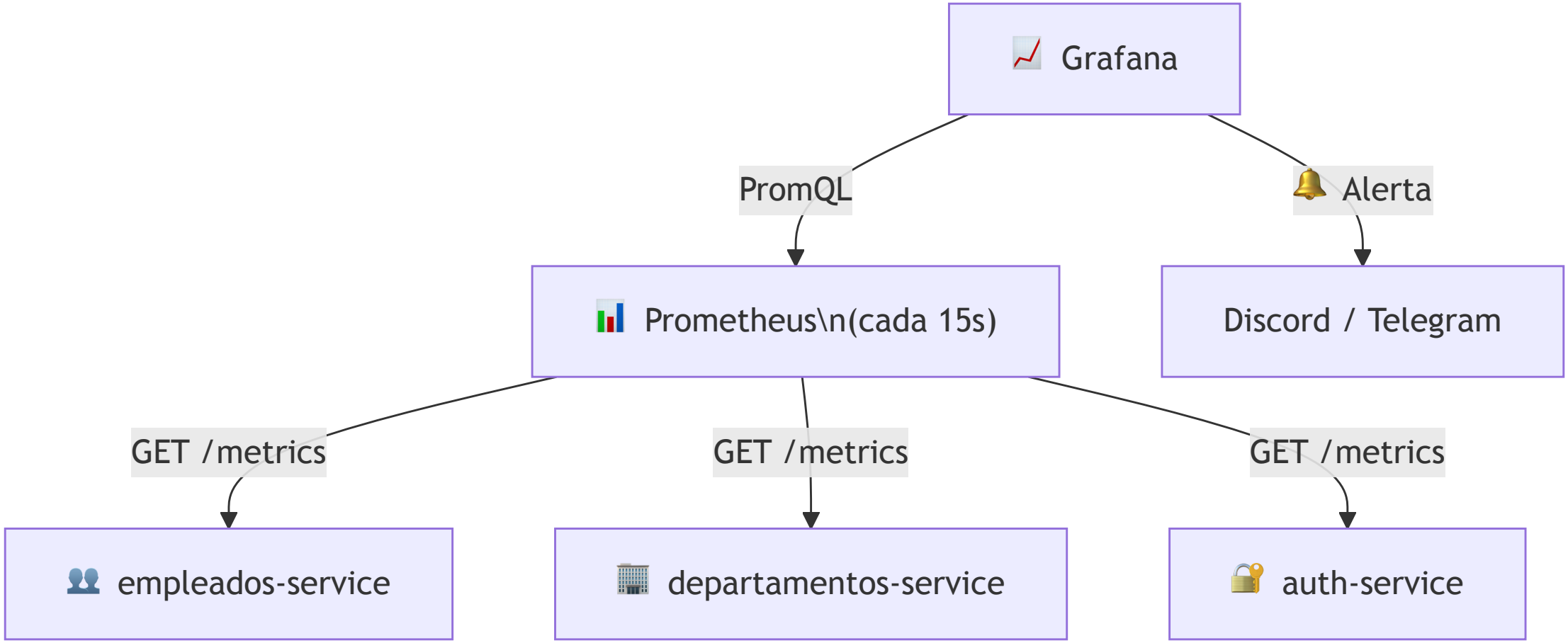
Patrón: Health Check por Terceros



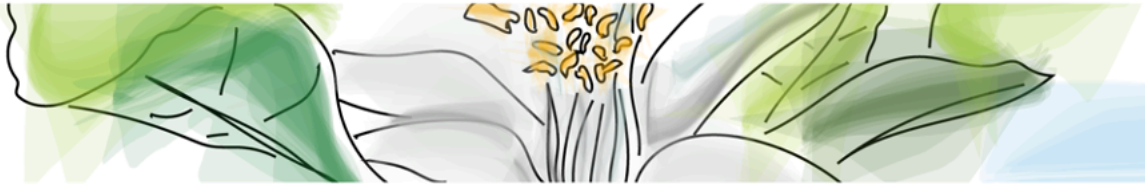
Health Check por Terceros

- **Problema**: ¿Cómo garantizar que los sistemas de monitoreo siempre sepan qué instancias están activas en entornos dinámicos (contenedores efímeros, auto-scaling)?
- **Posible solución**: Un componente externo (Prometheus, Service Registry) realiza el health check de forma activa y periódica en lugar de depender de que el servicio se auto-reporte.

Health Check por Terceros - Diagrama

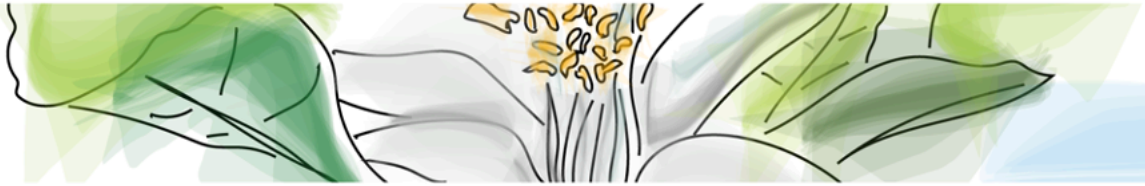


UNIQUINDÍO
en conexión territorial

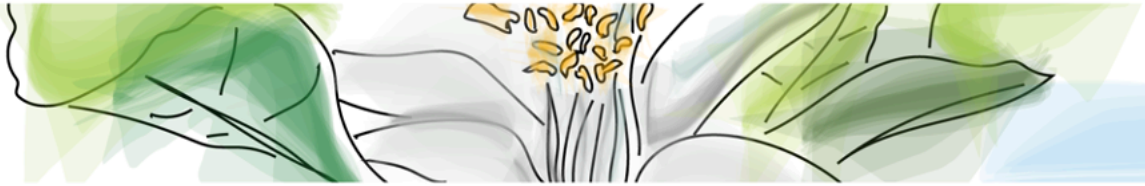


Health Check por Terceros - Beneficios y Dificultades

- **Beneficios**
  - Visibilidad centralizada de todos los servicios desde un único scraper
  - El servicio no necesita saber a quién reportar
  - Compatible con entornos dinámicos de contenedores
  - Prometheus acumula historial de métricas para análisis de tendencias
- **Dificultades**
  - El scraper debe tener acceso de red a todos los servicios
  - En entornos muy dinámicos requiere Service Discovery integrado
  - La configuración de redes Docker debe ser cuidadosa



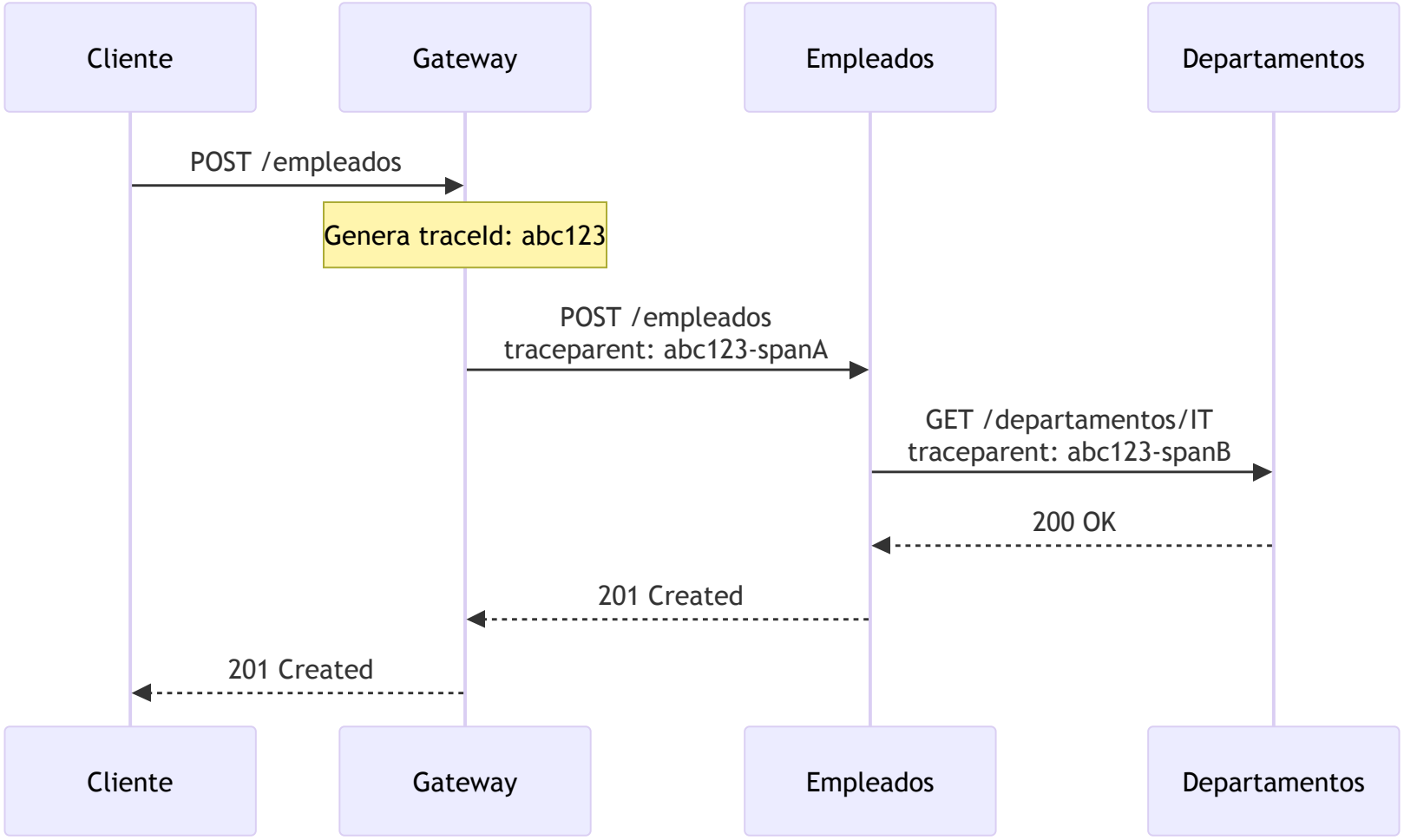
Patrón: Distributed Tracing



Distributed Tracing

- **Problema**: Una petición atraviesa múltiples servicios. Cuando hay lentitud o fallo, ¿cómo identificar exactamente qué servicio es el responsable?
- **Posible solución**: Instrumentar cada microservicio para generar y propagar un `traceId` único a través de cabeceras HTTP. Recopilar los spans en un servidor centralizado (Zipkin, Jaeger) para visualización.

Distributed Tracing - Propagación del contexto



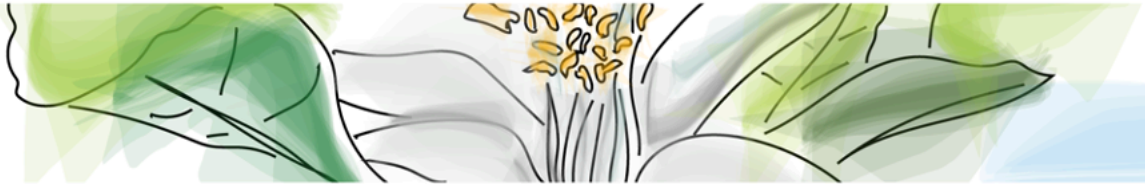
UNIQUINDÍO
en conexión territorial

El estándar **W3C Trace Context** define el formato de las cabeceras de propagación:

```
traceparent: 00-abc123def456789-spanIdABC-01
             ^^  ^^^^^^^^^^^^^^^  ^^^^^^^^  ^^
             |   traceId (16B)    spanId(8B)  flags
             versión
```

- `traceparent` : Identificador de la traza y el span actual
- `tracestate` : Metadata adicional del proveedor

> **Clave**: Gracias a este estándar, un servicio en **Python** y otro en **Node.js** pueden participar en la **misma traza** sin problemas de compatibilidad.



Distributed Tracing - Beneficios y Dificultades

- **Beneficios**
  - Visibilidad completa del flujo de una petición en sistemas distribuidos
  - Identifica exactamente qué servicio causa latencia o errores
  - Agnóstico al lenguaje gracias a estándares como W3C Trace Context
  - Correlaciona logs, métricas y trazas mediante el `traceId`
- **Dificultades**
  - Requiere instrumentar todos los servicios (incluyendo el broker de mensajes)
  - Overhead de red al propagar cabeceras y enviar spans al servidor
  - Configuración adicional por lenguaje/framework

¿Por qué **no** capturar el 100% de las trazas en producción?

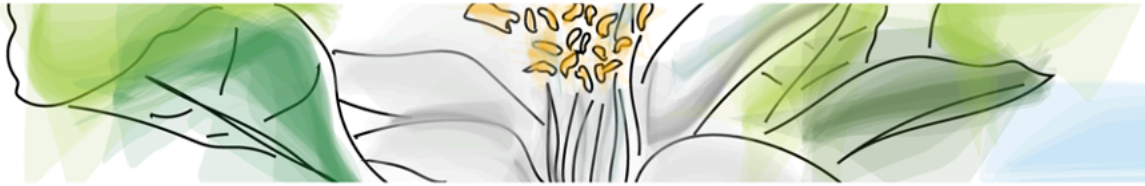
> 1.000 req/s × 10 servicios × N spans = millones de spans/segundo → costo insostenible

Tipo	Cuándo decide	Ventaja	Desventaja
Head-based	Al inicio de la traza	Simple, bajo overhead	Puede descartar trazas con errores
Tail-based	Al final (conoce el resultado)	Conserva errores y trazas lentas	Requiere OTel Collector

> **Regla práctica**: 100% de sampling en desarrollo y QA. En producción: head-based al 10-20%, o tail-based conservando **siempre** las trazas con errores (status code ≥ 500).



Patrón: Application Metrics



Application Metrics

- **Problema**: ¿Cómo medir de forma continua el comportamiento y rendimiento de cada microservicio para detectar anomalías y tomar decisiones basadas en datos?
- **Posible solución**: Instrumentar cada microservicio para exponer métricas clave en un formato estándar (Prometheus). Un agente externo las recolecta y almacena para análisis y visualización.

Google SRE define las **4 Golden Signals** como las métricas más importantes:

Signal	Métrica	Pregunta
Latency	Tiempo de respuesta	¿Cuánto tarda en responder?
Traffic	Peticiones por segundo	¿Cuánta carga recibe?
Errors	Tasa de errores	¿Cuántas peticiones fallan?
Saturation	Uso de CPU/Memoria	¿Qué tan lleno está?

Application Metrics - Librerías por lenguaje

Lenguaje	Librería	Endpoint
Java / Spring Boot	`micrometer-registry-prometheus`	`/actuator/prometheus`
Node.js	`prom-client`	`/metrics`
Python / FastAPI	`prometheus-fastapi-instrumentator`	`/metrics`
Python / Flask	`prometheus-flask-exporter`	`/metrics`
Go	`prometheus/client_golang`	`/metrics`
.NET	`prometheus-net.AspNetCore`	`/metrics`

Application Metrics - Métricas personalizadas

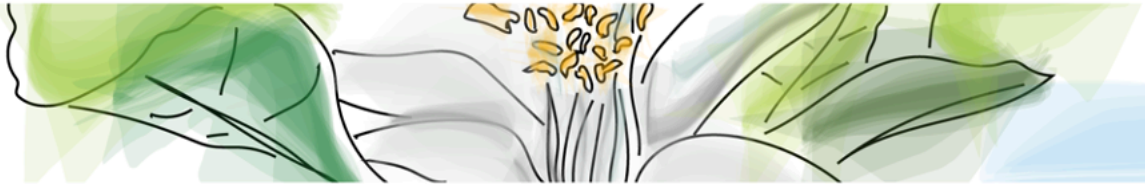
Además de las métricas automáticas (HTTP, JVM, etc.), se pueden definir métricas de negocio:

```python
# Python - prom-client
from prometheus_client import Counter, Histogram

empleados_creados = Counter(
    'empleados_creados_total',
    'Total de empleados creados exitosamente'
)

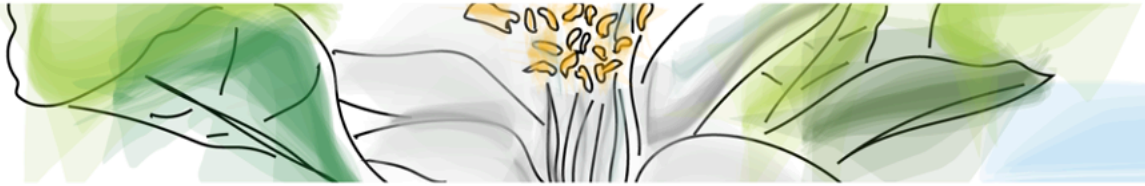
duracion_onboarding = Histogram(
    'onboarding_duration_seconds',
    'Duración del proceso de onboarding'
)

# Al crear un empleado:
empleados_creados.inc()
```

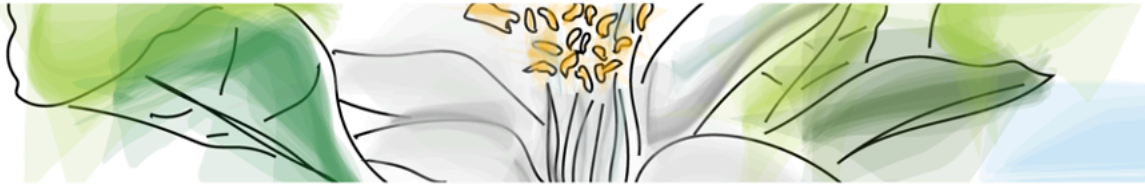


Application Metrics - Beneficios y Dificultades

- **Beneficios**
  - Base para alertas proactivas (si la tasa de errores supera X%, notificar)
  - Permite análisis de tendencias y capacity planning
  - Las métricas de negocio revelan comportamientos que las técnicas no muestran
- **Dificultades**
  - Demasiadas métricas generan ruido y costo de almacenamiento
  - Las métricas de alta cardinalidad pueden saturar Prometheus
  - Requiere definir con cuidado qué medir y con qué granularidad



Patrón: Audit Logging



Audit Logging

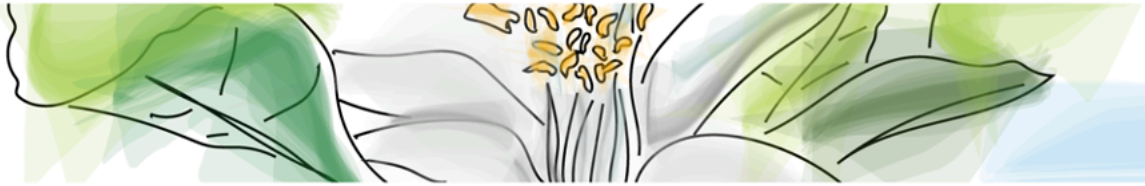
- **Problema**: ¿Cómo saber quién realizó qué acción y cuándo en un sistema distribuido? Especialmente relevante para cumplimiento normativo (GDPR, SOC2, etc.).
- **Posible solución**: Implementar un log de auditoría estructurado que registre cada acción significativa (creaciones, modificaciones, eliminaciones) con el contexto del actor que la realizó.

Audit Logging - Estructura de un evento de auditoría

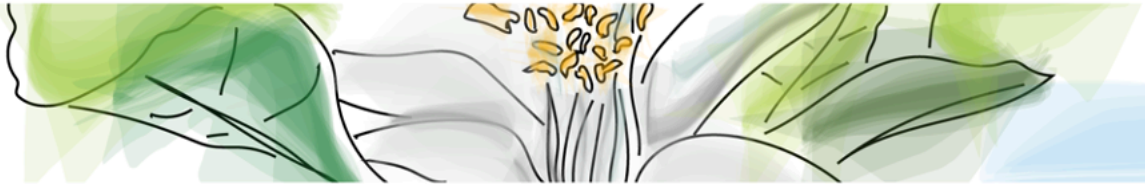
```
{
  "eventId": "uuid-aqui",
  "timestamp": "2026-04-21T20:00:00Z",
  "action": "EMPLEADO_CREADO",
  "actor": {
    "userId": "admin@empresa.com",
    "role": "ADMIN",
    "ip": "192.168.1.10"
  },
  "resource": {
    "type": "Empleado",
    "id": "E001"
  },
  "traceId": "abc123def456",
  "result": "SUCCESS"
}
```

Audit Logging - Beneficios y Dificultades

- **Beneficios**
  - Trazabilidad completa para cumplimiento normativo
  - Permite reconstruir la historia de un recurso
  - Detecta comportamientos anómalos (ej. un usuario que borra múltiples registros)
  - Correlacionable con trazas distribuidas vía `traceId`
- **Dificultades**
  - Volumen alto de datos que requiere retención y archivado
  - En microservicios, los audit logs están dispersos entre servicios
  - Requiere centralización para ser útil



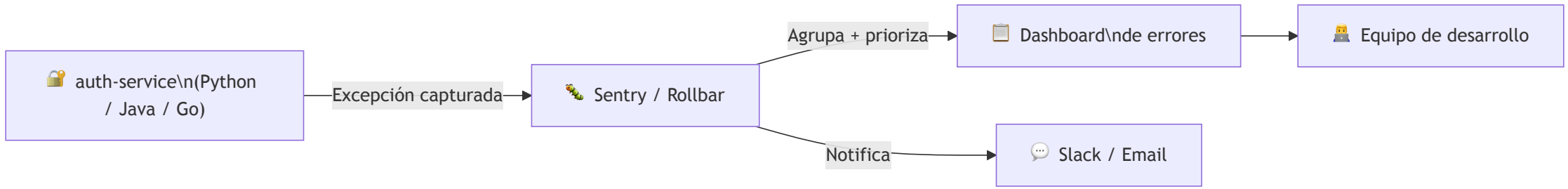
Patrón: Exception Tracking



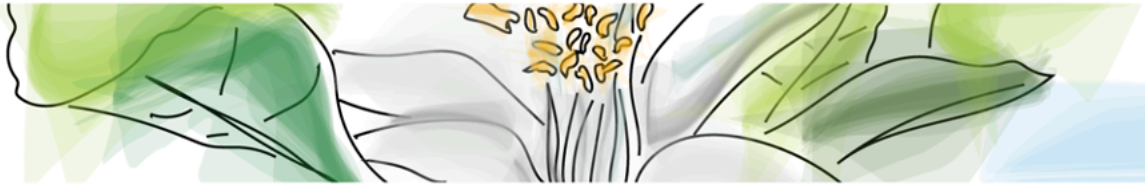
Exception Tracking

- **Problema**: En producción, las excepciones ocurren. ¿Cómo capturarlas, agruparlas y priorizarlas sin revisar manualmente miles de líneas de log?
- **Posible solución**: Integrar una herramienta de rastreo de excepciones (Sentry, Rollbar, etc.) que capture automáticamente las excepciones no manejadas, las agrupe por tipo y notifique al equipo.

Exception Tracking - Flujo

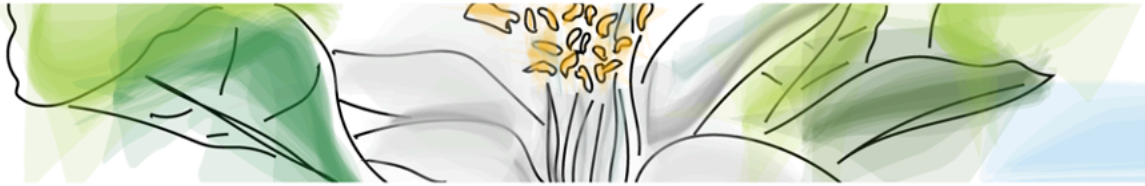


UNIQUINDÍO
en conexión territorial



Exception Tracking - Beneficios y Dificultades

- **Beneficios**
  - Detección automática de errores sin revisar logs manualmente
  - Agrupación inteligente de errores similares
  - Muestra frecuencia, usuarios afectados y contexto completo
  - Priorización por impacto real
- **Dificultades**
  - Herramientas como Sentry tienen costo en entornos de producción
  - Requiere instrumentación en cada servicio (por lenguaje)
  - Riesgo de capturar datos sensibles en el contexto de la excepción

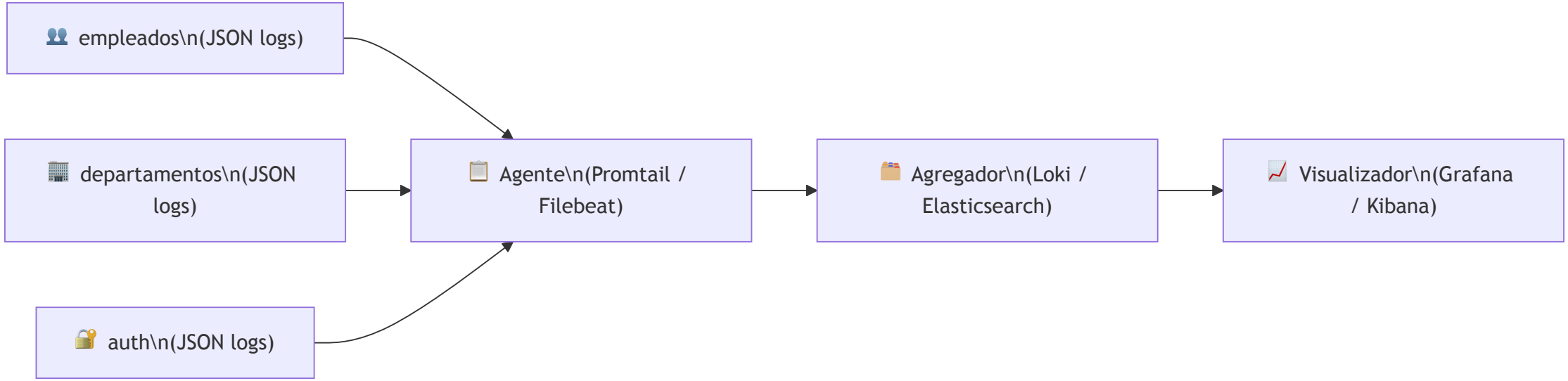


Patrón: Log Aggregation

Log Aggregation

- **Problema**: Los logs en microservicios están dispersos en N contenedores. Para depurar un error, ¿hay que conectarse a cada contenedor manualmente?
- **Posible solución**: Un agente recolector captura los logs de todos los contenedores y los envía a un sistema centralizado donde pueden buscarse, filtrarse y correlacionarse con métricas y trazas.

Log Aggregation - Flujo



UNIQUINDÍO
en conexión territorial

Log Aggregation - Stacks comparados

Stack	Componentes	Complejidad	Ideal para
Loki	Loki + Promtail + Grafana	Baja ✅	Integración con stack Prometheus/Grafana
ELK	Elasticsearch + Logstash + Kibana	Alta	Búsqueda full-text avanzada
Graylog	Graylog + MongoDB + Elasticsearch	Media	Equipos que prefieren UI dedicada

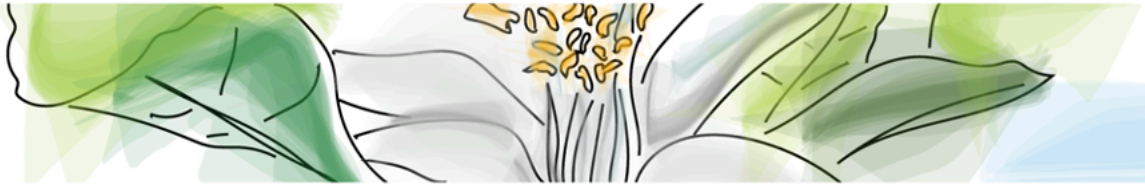
> **Recomendación académica**: Loki por su integración nativa con Grafana y menor consumo de recursos. En producción a escala, ELK ofrece más capacidades de búsqueda.

UNIQUINDÍO
en conexión territorial

Convención estándar de severidad (de menor a mayor):

Nivel	Uso correcto
`DEBUG`	Detalles de ejecución para desarrollo. **No usar en producción**
`INFO`	Eventos normales del flujo de negocio (empleado creado, sesión iniciada)
`WARN`	Situaciones inesperadas pero recuperables
`ERROR`	Fallos que requieren atención. Siempre incluir el stack trace
`FATAL`	El servicio no puede continuar. Implica reinicio

> **Regla de oro**: Si no sabes qué nivel usar, pregúntate: *"¿necesito despertar a alguien a las 3am por esto?"*



Log Aggregation - Beneficios y Dificultades

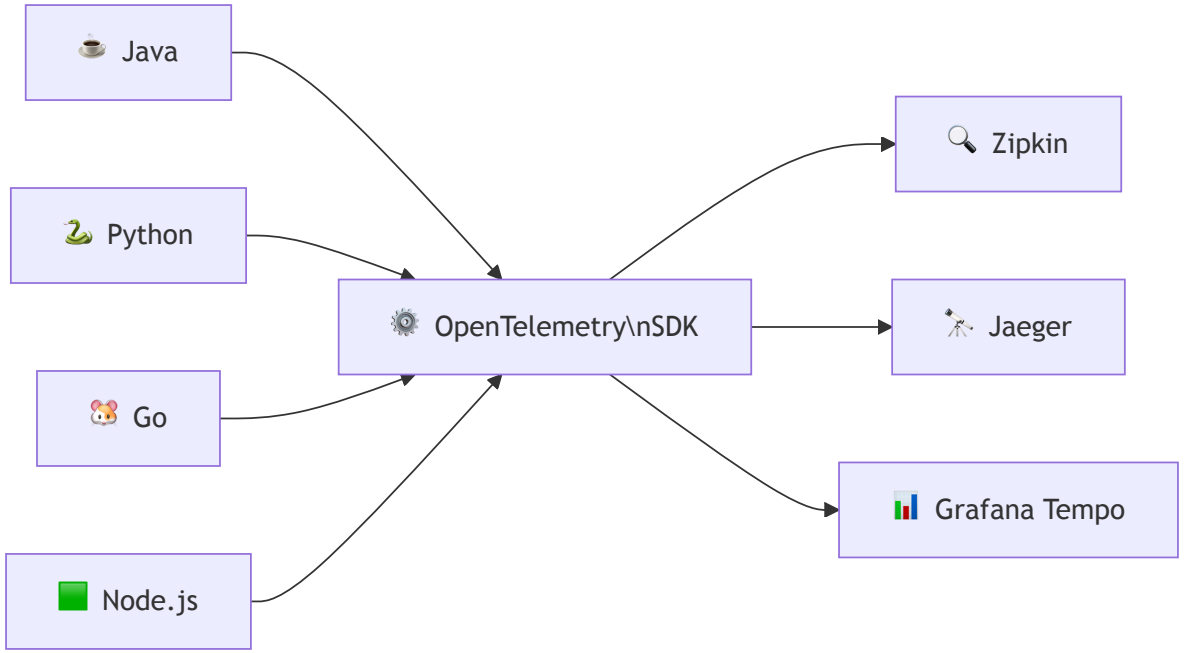
- **Beneficios**
  - Buscar logs de todos los servicios desde un solo lugar
  - Correlacionar con trazas y métricas mediante el `traceId`
  - Detectar patrones de error entre múltiples servicios
  - Auditoría y cumplimiento normativo centralizado
- **Dificultades**
  - Volumen de logs puede ser muy alto (requiere política de retención)
  - Logs no estructurados son difíciles de parsear automáticamente
  - Latencia entre emisión del log y disponibilidad en el sistema central



OpenTelemetry: El Estándar Abierto

¿Qué es OpenTelemetry?

- **Proyecto de la CNCF** (Cloud Native Computing Foundation)
- Estándar abierto y agnóstico para instrumentar sistemas distribuidos
- Unifica la recolección de **métricas, logs y trazas** bajo una sola API
- SDKs disponibles para **todos los lenguajes principales**

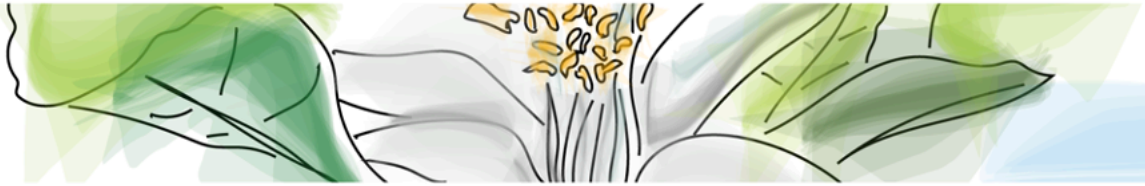


UNIQUINDÍO
en conexión territorial

Componente	Rol
API	Interfaz de instrumentación (qué medir)
SDK	Implementación concreta (cómo medir)
Collector	Agente/proxy que recibe, procesa y exporta telemetría
Exporters	Conectores hacia backends (Zipkin, Jaeger, Prometheus...)

Variables de entorno estándar:

```
OTEL_SERVICE_NAME=empleados-service
OTEL_EXPORTER_OTLP_ENDPOINT=http://otel-collector:4317
OTEL_PROPAGATORS=tracecontext,baggage
```



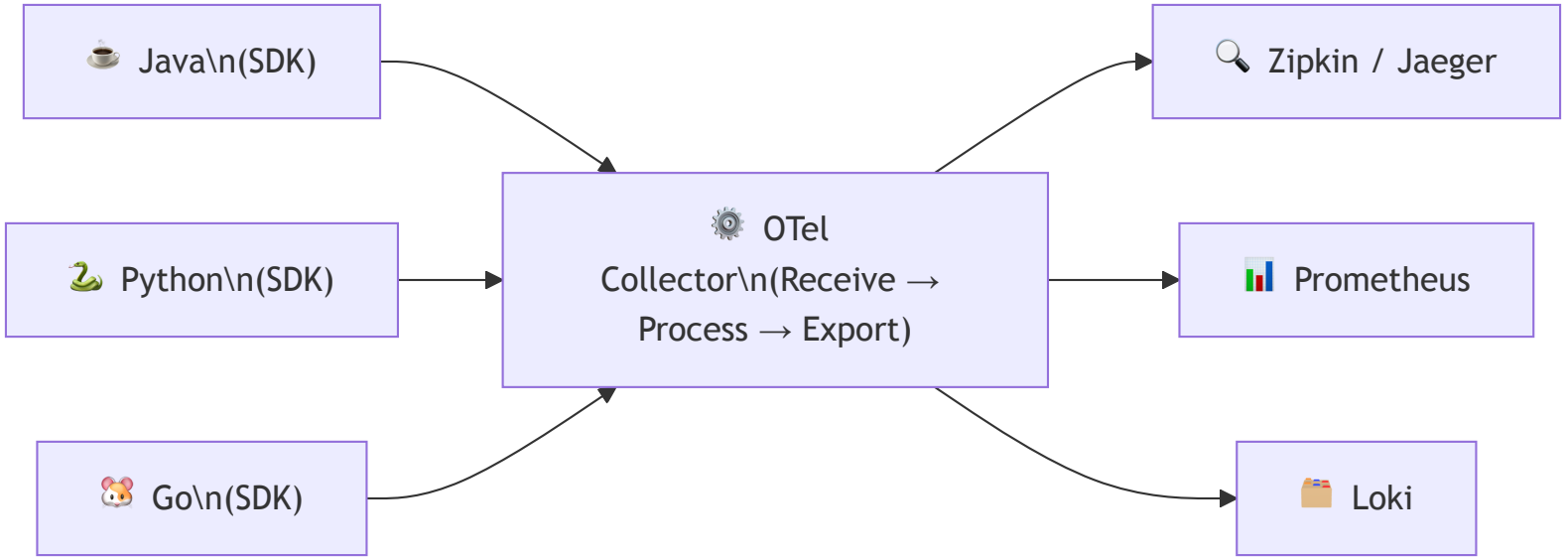
OpenTelemetry - SDKs por lenguaje

Lenguaje	SDK / Librería
Java	`opentelemetry-spring-boot-starter`
Node.js	`@opentelemetry/sdk-node`
Python	`opentelemetry-sdk` + `opentelemetry-instrumentation-fastapi`
Go	`go.opentelemetry.io/otel`
.NET	`OpenTelemetry.Extensions.Hosting`

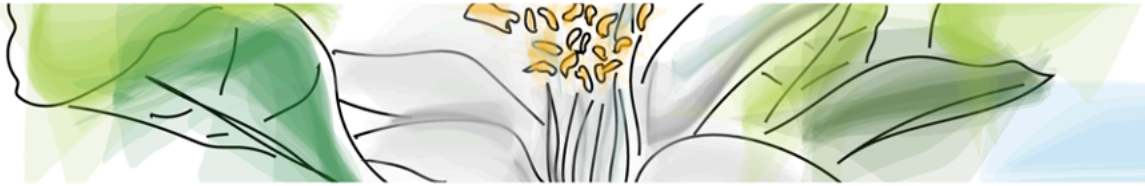
> **Ventaja clave**: Con OTel, un trace puede cruzar de un servicio Java a uno Python a uno Go manteniendo el mismo `traceId`, gracias al estándar W3C Trace Context.

OpenTelemetry Collector: El Middleware

El **OTel Collector** es un agente independiente (sidecar o gateway) que actúa como middleware de telemetría:

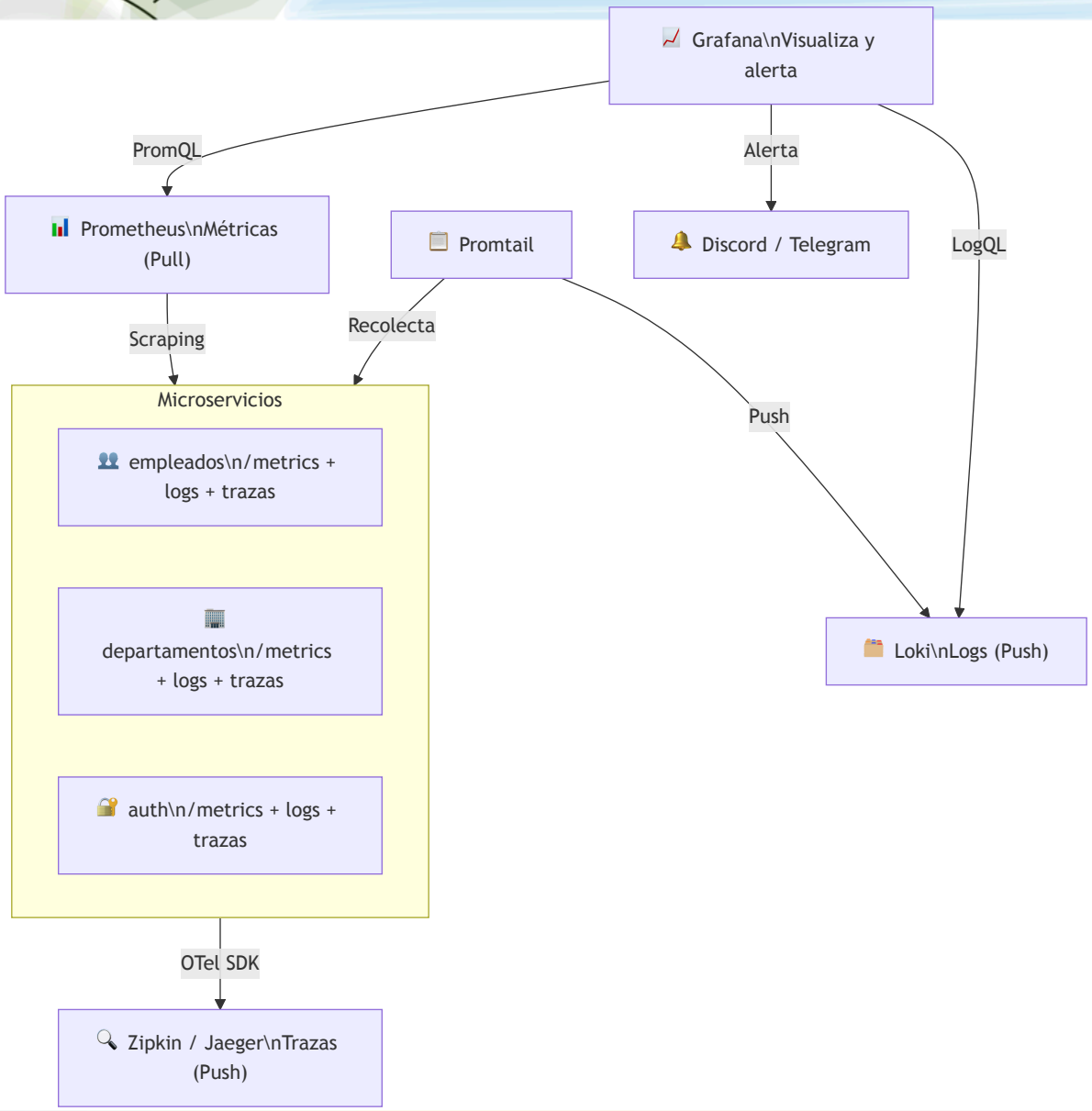


> **Beneficio clave**: Los servicios envían telemetría **a un solo punto**. El Collector decide a qué backends reenviarla, aplica filtros, enriquecimiento y tail-based sampling sin tocar el código de los servicios.



Herramientas del Ecosistema

Stack de Observabilidad Completo



UNIQUINDÍO
en conexión territorial

- **Rol**: Recolector y almacén de métricas (modelo Pull)
- **Lenguaje de consulta**: PromQL
- **Almacenamiento**: Time-series database (TSDB)

Queries PromQL fundamentales:

```
# Tasa de peticiones por segundo
rate(http_requests_total[1m])

# Latencia promedio
rate(http_duration_seconds_sum[1m])
/ rate(http_duration_seconds_count[1m])

# Porcentaje de errores 5xx
sum(rate(http_requests_total{status=~"5.."}[1m]))
/ sum(rate(http_requests_total[1m])) * 100
```

- **Rol**: Visualización de métricas y gestión de alertas
- **Conecta con**: Prometheus, Loki, Tempo, Elasticsearch y más
- **Características clave**:
  - Dashboards configurables con múltiples tipos de panel
  - **Alerting** integrado con canales externos
  - Aprovisionamiento automático vía archivos YAML/JSON

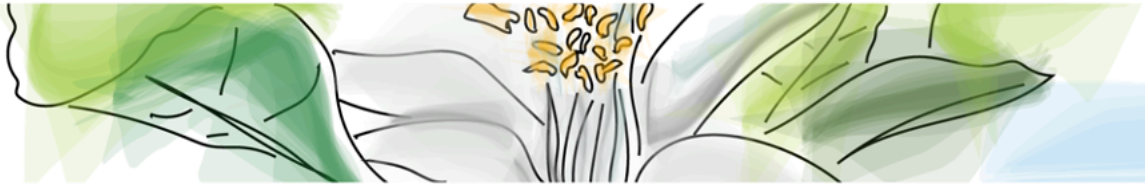
Tipos de panel más usados:

Panel	Uso
Time Series	Evolución de métricas en el tiempo
Stat	Valor actual (estado UP/DOWN)
Gauge	Porcentaje de uso
Table	Datos tabulares comparativos

Zipkin vs. Jaeger

Aspecto	Zipkin	Jaeger
Origen	Twitter	Uber
UI	Simple y directa	Más completa
Protocolo	Zipkin B3 / OTel	OTel / Jaeger nativo
Escalabilidad	Básica	Alta (Cassandra, Elasticsearch)
Ideal para	Entornos locales / académicos	Producción a escala
Puerto	9411	16686 (UI), 4317 (OTLP)

> **Recomendación académica**: Zipkin por su simplicidad. En producción, Jaeger o Grafana Tempo.



Alertas Proactivas

¿Por qué son necesarias las alertas?

```
Dashboard sin alertas:
  ✅ Tienes visibilidad
  ❌ Alguien tiene que estar mirando el dashboard

Dashboard con alertas:
  ✅ Tienes visibilidad
  ✅ El sistema te avisa cuando algo falla
  ✅ El equipo puede actuar antes de que el usuario lo reporte
```

> **Objetivo**: Pasar de un modelo **reactivo** (el cliente reporta el error) a uno **proactivo** (el sistema avisa antes de que el cliente lo note).

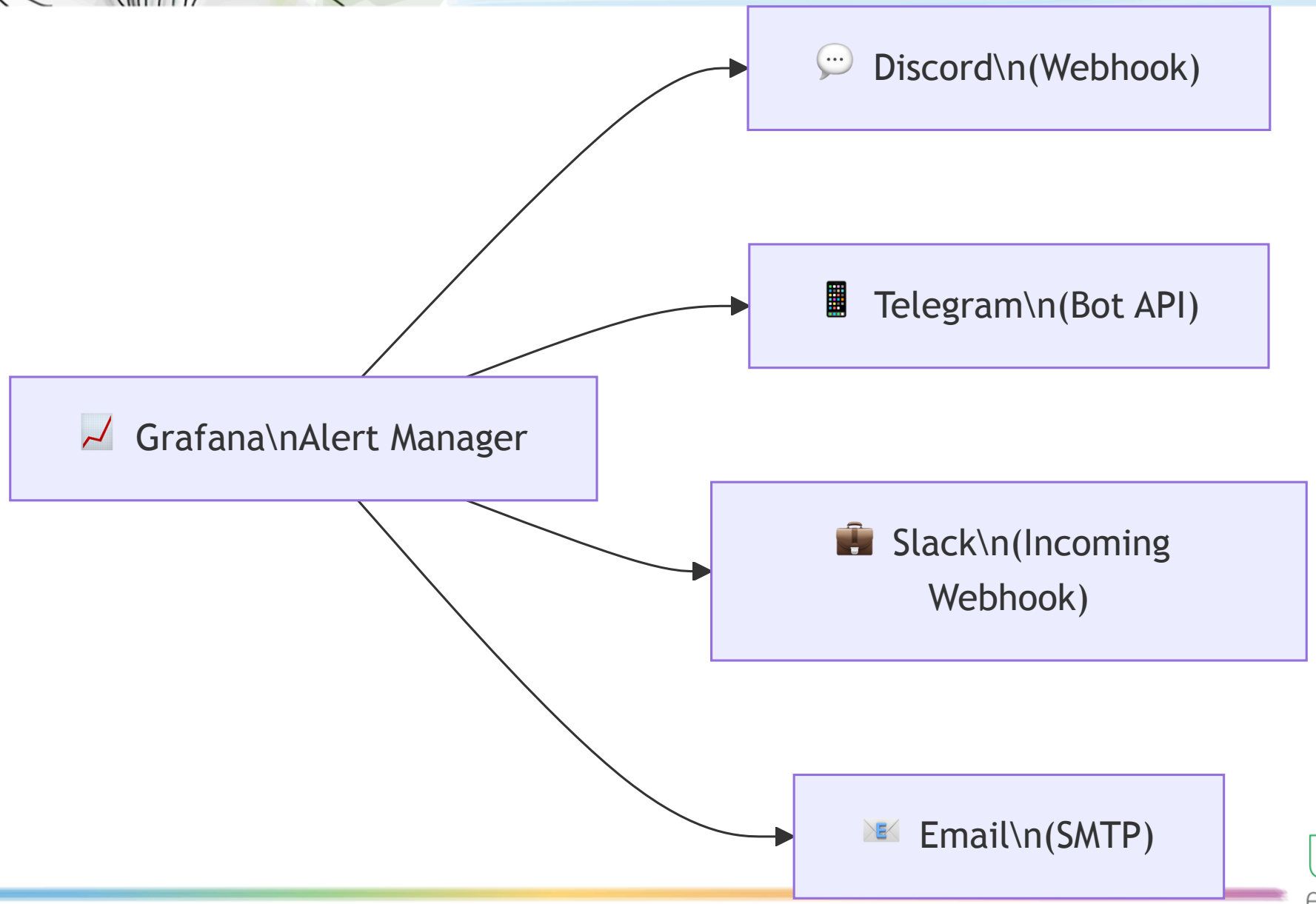
Alertas en Grafana - Componentes

Componente	Descripción
Alert Rule	Condición que se evalúa periódicamente sobre una query PromQL
Contact Point	Destino de la notificación (Discord, Telegram, Slack, Email)
Notification Policy	Regla que conecta alertas con contact points
Silences	Supresión temporal de alertas (ej. durante mantenimiento)

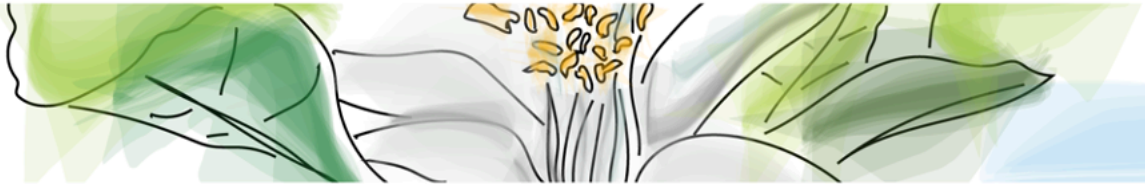
Alertas en Grafana - Reglas recomendadas

Regla	Condición PromQL	Severidad
Servicio caído	`up == 0` por 1 min	🔴 Critical
Alta tasa de errores	Errores 5xx > 10% por 2 min	🔴 Critical
Alta latencia	Latencia promedio > 2s por 2 min	🟡 Warning
Alto uso de memoria	Memoria > 85% por 5 min	🟡 Warning

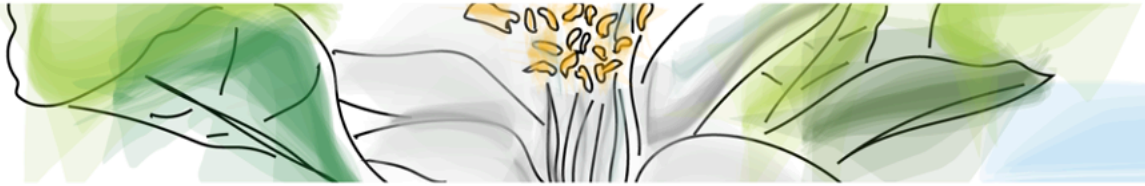
Alertas - Canales de notificación



UNIQUINDÍO
en conexión territorial

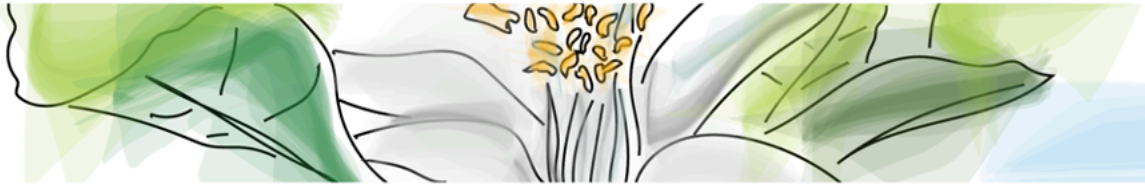


Mejores Prácticas y Antipatrones



Mejores Prácticas

1. **Instrumentar desde el inicio**: La observabilidad no es un añadido; diseñarla desde el primer servicio.
2. **Logs estructurados siempre**: JSON con `traceId` en cada línea de log.
3. **Usar estándares**: OpenTelemetry y W3C Trace Context para garantizar compatibilidad entre lenguajes.
4. **Golden Signals primero**: Medir latencia, tráfico, errores y saturación antes que métricas personalizadas.
5. **Aprovisionar dashboards en código**: Exportar dashboards de Grafana a JSON y versionarlos en Git.
6. **Probar las alertas**: Simular fallos para verificar que las alertas se disparan correctamente.



Antipatrones a evitar

Antipatrón	Problema
Logs sin estructura	Imposibles de parsear y correlacionar automáticamente
Sin `traceId` en los logs	No se puede correlacionar con la traza distribuida
Health check vacío	`{ "status": "OK" }` sin verificar dependencias reales
Demasiadas métricas	Alta cardinalidad satura Prometheus
Dashboard sin alertas	Monitoreo reactivo que depende de que alguien lo mire
Secrets en logs	Tokens o contraseñas que aparecen en registros de error
Ignorar el overhead	Tracing al 100% en producción puede impactar el rendimiento

Resumen: Patrones de Observabilidad

Patrón	Pilar	Problema que resuelve
Health Check API	🟢 Disponibilidad	¿Está el servicio listo para recibir tráfico?
Health Check por Terceros	🟢 Disponibilidad	Visibilidad centralizada en entornos dinámicos
Distributed Tracing	🔗 Trazas	¿Por dónde pasó y dónde falló la petición?
Application Metrics	📊 Métricas	¿Cómo se está comportando el sistema? (Golden Signals)
Audit Logging	📋 Logs	¿Quién hizo qué y cuándo? (trazabilidad de negocio)
Exception Tracking	📋 Logs	Captura y priorización automática de errores
Log Aggregation	📋 Logs	Centralizar logs dispersos para búsqueda y correlación

en conexión territorial



UNIVERSIDAD DEL QUINDÍO ®

ALTA CALIDAD
Res.MEN 014915 - 02 AGO 2022

CERTIFICADO POR SU COMPROMISO RESPONSABILIDAD SOCIAL FENALCO SOLIDARIO ®

Arquitectura Basada en Microservicios

Programa de Ingeniería de Sistemas y Computación
Universidad del Quindío

CC BY NC SA 2026 Christian Andrés Candela Uribe